

```
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
```

```
#define MAXPAROLA 30
#define MAXRIGA 80
```

```
int main(int argc, char *argv[])
{
    int freq[MAXPAROLA]; /* vettore di contatori
delle frequenze delle lunghezze delle parole */
    char riga[MAXRIGA];
    int i, inizio, lunghezza;
    FILE *f;
```

```
for(i=0; i<MAXPAROLA; i++)
    freq[i]=0;
```

```
if(argc != 2)
```

```
{
    fprintf(stderr, "ERRORE, serve un parametro con il nome del file\n");
    exit(1);
}
```

```
f = fopen(argv[1], "r");
if(f==NULL)
```

```
{
    fprintf(stderr, "ERRORE, impossibile aprire il file %s\n", argv[1]);
    exit(1);
}
```

```
while( fgets( riga, MAXRIGA, f ) != NULL )
```



Advanced IO

Multiplexing IO

Stefano Quer

Dipartimento di Automatica e Informatica

Politecnico di Torino

License Information

This work is licensed under the license



Attribution-NonCommercial-NoDerivatives 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to copy and distribute the material in any medium or format in unadapted form and for noncommercial purposes only.

① **BY:** Credit must be given to you, the creator.

② **NC:** Only noncommercial use of your work is permitted.

Noncommercial means not primarily intended for or directed towards commercial advantage or monetary compensation.

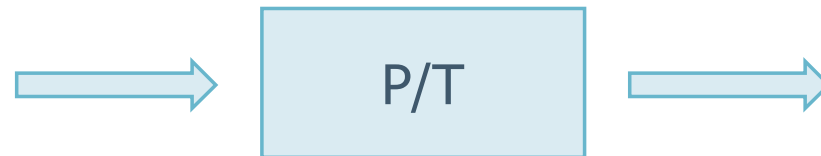
③ **ND:** No derivatives or adaptations of your work are permitted.

To view a copy of the license, visit:

<https://creativecommons.org/licenses/by-nc-nd/4.0/?ref=chooser-v1>

Problem definition

- ❖ In many applications
 - We read from one descriptor
 - We write to another
 - We start the process all-over again



- ❖ IO can be performed using blocking, non blocking, or asynchronous IO

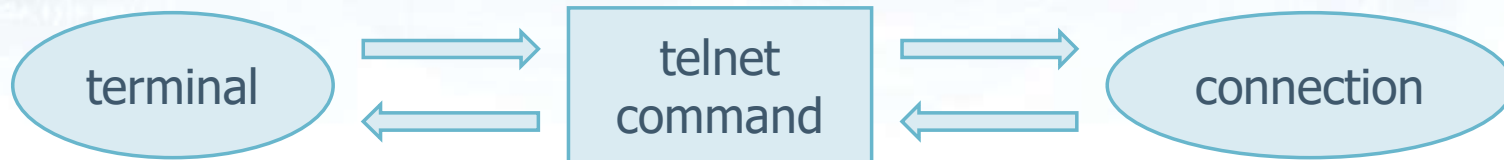
```
while ((n = read(STDIN_FILENO, buf, BUFSIZ)) > 0)
    if (write(STDOUT_FILENO, buf, n) != n)
        err_sys("write error");
```

Problem definition

- ❖ There are applications in which we need to perform IO on different descriptors at the same time
- ❖ How can we read (write) from (to) **two** separate descriptors?



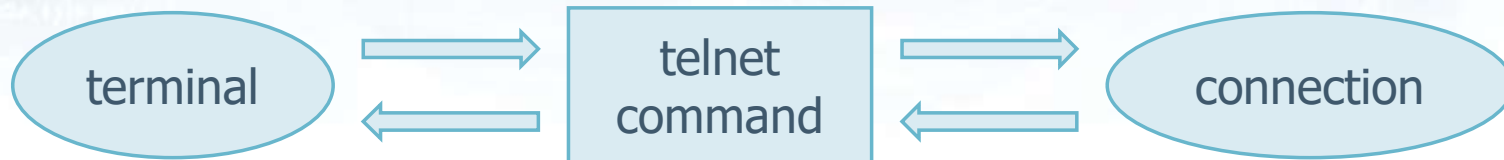
Problem definition: An Example



❖ A telnet command

- Reads from the terminal (standard input) and write to a network connection
- Reads from the network connection and write to the terminal (standard output)

Problem definition: An Example



- We have two reading descriptors and we cannot do a blocking read on either descriptor

- Data may appear on **one** descriptor while we are blocked in a read on the **other**

- We can

- Use different processes or different threads
- Use non-blocking or asynchronous IO
- **Use IO multiplexing**

Solution 1

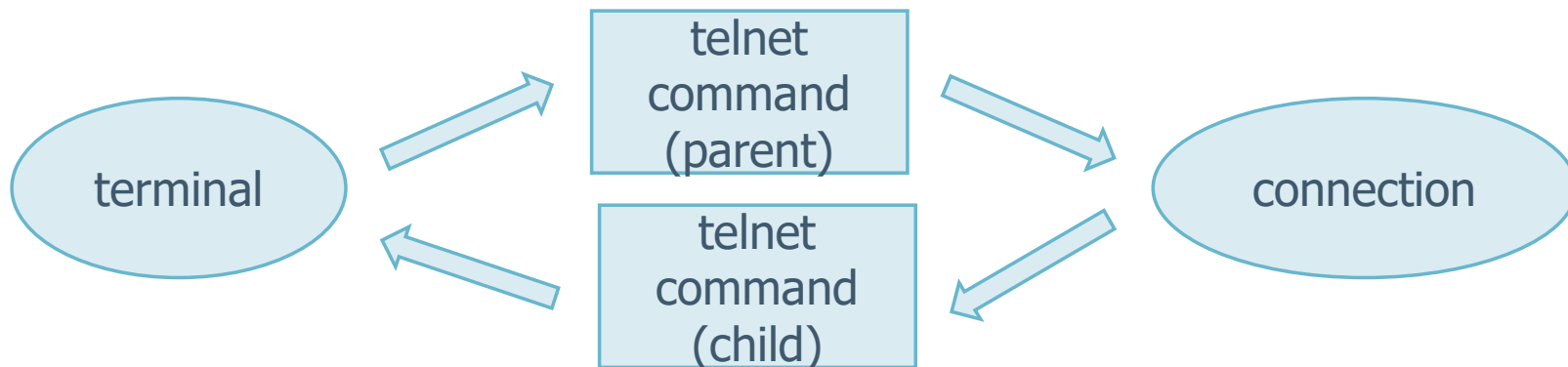
Solution 2

Solution 3

Solution 1: Multi-threading

❖ Multi-processing or multi-threading

- Run two threads (or fork the process) with the parent handling one direction of data and the child the other one



- Each thread (process) can perform blocking (or non blocking) reads

Pros and Cons

- ❖ We are forced to resort to multi-threading or multi-processing
 - We may need to synchronize the process or the threads
 - We may need to perform a clean termination when EOF is received in one direction we must stop both workers

Solution 2: Non-blocking IO

❖ Non-blocking (asynchronous) IO

- We use nonblocking IO in a single process
 - We set both descriptors to be nonblocking
 - We issue a read on the first descriptor
 - If data is present, we read it and process it
 - If there is no data to read, the call returns and we do the same on the second descriptor
 - We wait for some amount of time and then we try to read from the first descriptor again
- With asynchronous IO we avoid polling by letting the kernel notify us with a signal when a descriptor is ready

Pros and Cons

❖ Asynchronous IO is far to be easy to implement

- Portability can be an issue as different systems provide their own form of asynchronous IO
- With polling, we have to decide how often we want to poll
- With signals the number of signals allowed for each process is limited
 - POSIX allows us to select which signal to use for the notification
 - Anyway, the number of signals we can use is still far less than the number of possible open file descriptors

Solution 3: Multiplexing IO

- ❖ The multiplexing approach to IO
 - The idea is to use a function that
 - Accepts more than one IO descriptor
 - Doesn't return until one of the descriptors is ready for IO
 - On return, it tells us which descriptors are ready for IO

Solution 3

❖ The multiplexing IO logic flow

➤ We build a list of the descriptors that we are interested in

- Usually more than one descriptor
- We need to perform IO on each one of them

➤ We use a multiplexing IO function

- Three functions allow us to perform IO multiplexing
 - select, pselect, poll

Functions pselect and poll are variation of select

- They differ from the user interface point of view
- Include POSIX `<sys/select.h>`
 - Or `<sys/types.h>`, `<sys/time.h>`, and `<unistd.h>`

The select function

```
#include <sys/select.h>
int select (
    int maxfdp1,
    fd_set *restrict readfds,
    fd_set *restrict writefds,
    fd_set *restrict exceptfds,
    struct timeval *restrict tvptr
);
```

- ❖ Function **select** allows IO multiplexing under all POSIX-compatible platforms
- ❖ The arguments tell the kernel
 - Which descriptors we are interested in
 - Which conditions we focus on (for each descriptor)
 - How long we want to wait

The select function

```
int select (    int maxfdp1,
               fd_set *restrict readfds,
               fd_set *restrict writefds,
               fd_set *restrict exceptfds,
               struct timeval *restrict tvptr );
```

Arguments	Meaning
int maxfdp1	Indicates the "maximum file descriptor plus 1" We compute the highest descriptor that we are interested in, considering all three of the descriptor sets, add 1.
fd_set *restrict readfds fd_set *restrict writefds fd_set *restrict exceptfds	Pointers to descriptor sets for reading, writing, check exceptions. Any of them can be null pointers if we are not interested. A descriptor set is stored in an fd_set data type. We can consider it to be just an array of bits.
struct timeval *restrict tvptr	Specifies how long we want to wait in terms of seconds and microseconds. If tvptr is NULL, we wait forever. If tvptr->tv_sec == 0 && tvptr->tv_usec == 0, we do not wait at all. This is a way to poll. If tvptr->tv_sec != 0 tvptr->tv_usec != 0, we wait the specified number of seconds and microseconds.

The select function

```
int select ( int maxfdp1,  
            fd_set *restrict readfds,  
            fd_set *restrict writefds,  
            fd_set *restrict exceptfds,  
            struct timeval *restrict tvptr );
```

- ❖ Objects of type `fd_set` data type can be manipulated by functions or macros

Arguments	Meaning
<code>void FD_SET (int fd, fd_set *fdset);</code>	Set to all zero bits, by <code>FD_ZERO</code>
<code>void FD_CLR (int fd, fd_set *fdset);</code>	Turn on a single bit in a set, by <code>FD_SET</code>
<code>void FD_ZERO (fd_set *fdset);</code>	Clear a single bit, by <code>FD_CLR</code>
<code>int FD_ISSET (int fd, fd_set *fdset);</code>	Test whether a given bit is turned on, by <code>FD_ISSET</code>

Example

- ❖ The following code show how to prepare a read set

```
fd_set rset;  
int fd;
```

We define the set

```
FD_ZERO (&rset);  
FD_SET (fd, &rset);  
FD_SET (STDIN_FILENO, &rset);
```

We zero the set

We set bits for the descriptors we are interested in

- ❖ Once, select returns, we test whether a given bit is still set

```
if (FD_ISSET(fd, &rset)) {  
    ...  
}
```

The select function

```
int select ( int maxfdp1,  
             fd_set *restrict readfds,  
             fd_set *restrict writefds,  
             fd_set *restrict exceptfds,  
             struct timeval *restrict tvptr );
```

❖ There are three possible return values

Return value	Meaning
-1	An error has occurred.
0	No descriptors are ready. This happens if the time limit expires before any of the descriptors are ready. When this happens, all the descriptor sets will be zeroed out
>0	Indicates the number of descriptors that are ready . This value is the sum of the number of descriptors ready in all three sets. If the same descriptor is ready to be read and written, it will be counted twice in the return value. The only bits left on in the three descriptor sets are the bits corresponding to the descriptors that are ready

The select function

```
int select ( int maxfdp1,  
            fd_set *restrict readfds,  
            fd_set *restrict writefds,  
            fd_set *restrict exceptfds,  
            struct timeval *restrict tvptr );
```

- ❖ A descriptor is consider as “ready”
 - If a read from that descriptor won’t block (if it is in the read set, readfds)
 - If a write from that descriptor won’t block (if it is in the write set, writefds)
 - If an exception condition is pending on that descriptor (if it is in the exception set, exceptfds)
 - File descriptors for regular files always return ready for reading, writing, and exception conditions

Example

Wait on more than
one file descriptors

Set fd_set to zeros

```
fd_set set;  
FD_ZERO (&set);  
FD_SET (fd1, &set);  
FD_SET (fd2, &set);  
FD_SET (fd3, &set);  
size = max(fd1, fd2, fd3) + 1;  
while (1) {  
    // Block until at least ONE input arrives  
    if (select (size, &set, NULL, NULL, NULL) < 0) {  
        // handle error  
    }  
    for (i=0; i<size; ++i) {  
        if (FD_ISSET (i, &set)) {  
            // there is data for fd in position i  
        }  
    }  
}
```

Add fds to read fd_set

Set fd_set size to max fd + 1

Serve all fd with input pending